

"Melango"

an all-in-one learning management web application

Software Requirements Specification

Course: CSE470 – Software Engineering
Department of Computer Science and Engineering
BRAC University

Prepared by

Student ID	Name
23301489	Tahsin Pathan
23301334	Shakeel
22299386	Tabassum Islam
23101108	Ashrafi Tahmid

Version 1.0
September 2026

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Overview
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Features
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Constraints
 - 2.6 Assumptions and Dependencies
- 3. System Requirements
 - 3.1 Functional Requirements
 - 3.1.1 Authentication & Authorization
 - 3.1.2 Course & Enrollment Management
 - 3.1.3 Academic Content & Assessment
 - 3.1.4 Communication, Tracking & Additional Services
 - 3.2 Non-Functional Requirements
 - 3.2.1 Performance Requirements
 - 3.2.2 Security Requirements
 - 3.2.3 Reliability & Availability
 - 3.2.4 Maintainability
 - 3.2.5 Scalability
 - 3.3 External Interface Requirements
 - 3.3.1 User Interfaces
 - 3.3.2 Hardware Interfaces
 - 3.3.3 Software Interfaces
 - 3.3.4 Communication Interfaces
- 4. Technology Stack & Architectural Overview
 - 4.1 MERN Stack Components
- 5. Tentative Development Plan (Agile Methodology)
- 6. Acceptance Criteria
- 7. Conclusion
- 8. Sprint Feature Descriptions

1. Introduction

1.1 Purpose

This Software Requirement Specification (SRS) document outlines the requirements for developing “Melango” – an all-in-one Learning Management System (LMS) web application using the MERN stack (MongoDB, Express.js, React.js, Node.js). The primary goal of this application is to provide a secure and efficient academic ecosystem in which students can join courses, complete assignments and quizzes, track progress, and earn achievements, while teachers manage content, grading, attendance, and communication, and administrators oversee users, courses, and platform reports.

1.2 Scope

The scope of this project includes the design, development, testing, and demonstration of the Melango LMS web application, catering to:

- Students who can register, enroll in courses by enrollment code, access materials, submit assignments, attempt quizzes, view progress, and receive certificates.
- Teachers who can create courses, upload materials, post assignments and quizzes, grade submissions, take attendance, post announcements, and moderate discussions.
- Administrators who oversee user roles, courses, announcements, search, messaging, notifications, and high-level reports.
- Public visitors who can browse the marketing site, course catalog, course previews, blog, and contact pages before signing up.

This SRS covers the functional and non-functional requirements for the project, the constraints, assumptions, high-level design, and the development plan following Agile methodology. Feature screenshots are omitted from this version; the implemented product itself is the visual reference.

1.3 Definitions, Acronyms, and Abbreviations

- MERN: MongoDB, Express.js, React.js, Node.js.
- LMS: Learning Management System.
- JWT: JSON Web Token used for session authentication.
- RBAC: Role-Based Access Control.
- API: Application Programming Interface.
- REST: Representational State Transfer.
- SMTP: Simple Mail Transfer Protocol for outgoing email.
- KPI: Key Performance Indicator shown on role dashboards.

1.4 References

- MongoDB documentation – <https://www.mongodb.com/docs/>
- Express.js documentation – <https://expressjs.com/>
- React.js documentation – <https://react.dev/>

- Node.js documentation – <https://nodejs.org/docs/>
- Vite documentation – <https://vitejs.dev/>
- Stripe API documentation – <https://stripe.com/docs/api>
- IEEE 830-1998 Recommended Practice for Software Requirements Specifications (structural reference).
- WeHeal CSE470 Project SRS (template followed for document organization).

1.5 Overview

Section 2 provides an overall description of the product, including its user classes and environment. Section 3 details the functional and non-functional requirements. Section 4 outlines the technology stack and architectural overview. Section 5 presents the development plan, including sprint-wise breakdown and deliverables. Section 6 states acceptance criteria. Section 7 concludes the document. Section 8 describes implemented features in sprint order.

2. Overall Description

2.1 Product Perspective

Melango is a standalone LMS built on the MERN stack. The React client (Vite) communicates with an Express.js REST API. Persistent data is stored in MongoDB. If a local MongoDB instance is unavailable, the server can fall back to an in-memory MongoDB instance for demonstration. Optional external services include Stripe (payments; simulation mode when no secret key is configured) and SMTP email (console logging when SMTP is not configured). The product includes a public marketing website and authenticated role-based dashboards.

2.2 Product Features

Melango implements the following 20 core LMS features:

- 1. Role Management – Supports Student, Teacher, and Admin roles with role-based permissions.
- 2. Course Creation – Allows instructors to create and manage courses.
- 3. Course Enrollment – Students can join courses using enrollment codes.
- 4. Dashboard – Provides an overview of courses, activities, and deadlines.
- 5. Assignment Management – Create, schedule, and manage assignments.
- 6. Assignment Submission – Students can submit files, links, or text responses.
- 7. Online Grading System – Teachers can grade submissions digitally.
- 8. Feedback & Comments – Enables personalized feedback and communication.
- 9. Announcement Board – Share important notices and course updates.
- 10. Discussion Forum – Supports academic discussions and Q&A.
- 11. Quiz & Exam Module – Create and manage quizzes and examinations.
- 12. Attendance Tracking – Record and monitor student attendance.

- 13. Learning Materials Repository - Store notes, PDFs, videos, and learning resources.
- 14. Calendar & Deadline Tracker - Track academic events and deadlines.
- 15. Notification System - Send reminders and important alerts.
- 16. Messaging System - Direct communication between students and instructors.
- 17. Progress Analytics - Monitor academic performance and engagement.
- 18. Search & Filter System - Quickly locate courses, materials, and assignments.
- 19. Admin Control Panel - Manage users, permissions, courses, and platform reports.
- 20. Gamification System - Uses badges, achievements, points, and leaderboards to increase engagement.

2.3 User Classes and Characteristics

- Students: learners who enroll in courses and complete academic work. They need a simple workspace for courses, assignments, quizzes, materials, calendar, messages, notifications, progress, attendance, and achievements.
- Teachers: instructors who own courses. They need tools to create courses, publish materials, manage assignments and quizzes, grade work, post announcements, host discussions, take attendance, and communicate with students.
- Administrators: platform operators who manage users and roles, review courses, post system announcements, search records, and monitor reports. They have broader access than teachers or students.
- Public visitors: unauthenticated users exploring Melango before creating an account. They use the marketing site and catalog only.

2.4 Operating Environment

- Web application: accessible on modern browsers (Chrome, Firefox, Edge, Safari).
- Mobile compatibility: responsive layout for tablets and smartphones.
- Client: React 19 application bundled with Vite, typically served on `http://localhost:5173` during development.
- Server: Node.js and Express.js API, configured for port 5001 in the project environment (default 5000 if unset).
- Database: MongoDB (local or cloud). In-memory MongoDB is used automatically when local MongoDB is unavailable.
- Hosting: suitable for deployment on Render, Railway, Vercel (client), AWS, or an equivalent Node-capable host.

2.5 Constraints

- Academic timeline: the project is delivered within a CSE470 semester by a team of four students.

- Security: passwords are hashed with bcrypt; API routes that mutate or read private data require JWT authentication; role checks restrict teacher/admin actions.
- Payment: full Stripe live processing is optional; the system may run in simulation mode without STRIPE_SECRET_KEY.
- Email: SMTP is optional; without credentials, emails are written to the server console.
- In-memory database: demo data created in memory is lost when the API process restarts.
- Browser-only client: no native mobile application is in scope.

2.6 Assumptions and Dependencies

- Users have a working internet connection and a modern browser.
- Node.js and npm are available on developer machines.
- If MongoDB is not installed locally, mongodb-memory-server can start a temporary database.
- Teachers will share enrollment codes with students so they can join courses.
- Third-party payment and email services are used only when keys are supplied.
- Demo accounts (student@melango.com, teacher@melango.com, admin@melango.com) may be seeded for evaluation.

3. System Requirements

3.1 Functional Requirements

3.1.1 Authentication & Authorization

1. Registration and Login

FR-1: The system shall allow a visitor to create an account with name, email, password, and role (student or teacher). Admin accounts are provisioned by seed data or an existing admin.

FR-2: The system shall reject registration when the email is already in use or required fields are invalid.

FR-3: The system shall authenticate users with email and password and return a JWT for subsequent API calls.

FR-4: The system shall deny login when credentials are incorrect or the account is inactive.

2. Role-Based Access Control (RBAC)

FR-5: The system shall provide different dashboards and navigation for students, teachers, and administrators.

FR-6: The system shall restrict course creation, assignment creation, grading, and user administration to authorized roles.

FR-7: The client shall redirect unauthenticated users away from /app routes to the login page.

3.1.2 Course & Enrollment Management

1. Course Management

FR-8: Teachers shall be able to create a course with a name, description, and generated enrollment code.

FR-9: Students and teachers shall be able to view courses they own or are enrolled in.

FR-10: Public visitors shall be able to browse a catalog of featured courses and open a course preview page.

FR-11: Administrators shall be able to view courses on the platform for oversight.

2. Enrollment

FR-12: Students shall be able to join a course by entering a valid enrollment code.

FR-13: The system shall prevent duplicate enrollment in the same course.

FR-14: Premium courses shall require a successful checkout (Stripe or simulation) before full access is granted, where configured.

3.1.3 Academic Content & Assessment

1. Learning Materials

FR-15: Teachers shall be able to add learning materials to a course they own.

FR-16: Enrolled students shall be able to view materials and mark a material as completed.

2. Assignment Management

FR-17: Teachers and admins shall be able to create an assignment with title, description, due date, and total marks for a course they own.

FR-18: Teachers shall be able to update or delete an assignment they own (API).

FR-19: Enrolled students shall be able to list assignments for a selected course, including due date and their submission status.

3. Assignment Submission

FR-20: Students shall be able to submit assignment work as text and an optional link.

FR-21: The API shall accept an optional file upload with a submission.

FR-22: Teachers shall be able to list submissions for an assignment and assign marks and comments.

FR-23: Students shall be able to see submission status and awarded marks after grading.

4. Quizzes

FR-24: Teachers shall be able to create quizzes for a course.

FR-25: Students shall be able to attempt an available quiz and receive a recorded result.

3.1.4 Communication, Tracking & Additional Services

1. Announcements and Discussions

FR-26: Teachers and admins shall be able to post announcements to a course or the platform.

FR-27: Enrolled users shall be able to create discussion threads and reply to threads in a course.

2. Messaging and Notifications

FR-28: Authenticated users shall be able to send and receive private messages with their contacts.

FR-29: The system shall create in-app notifications for relevant events (for example, a new assignment posted to enrolled students).

FR-30: Users shall be able to list notifications, mark one as read, and mark all as read.

3. Calendar and Deadline Tracker

FR-31: The system shall list calendar events for the current user, including stored events.

FR-32: The calendar shall include derived deadlines from assignments (due date) and quizzes (available-until date).

FR-33: Students, teachers, and admins shall be able to create a personal calendar event. Course-linked events may be created only by the course owner.

4. Attendance, Progress, Search, Reviews

FR-34: Teachers shall be able to record attendance for students in a course they own.

FR-35: Students shall be able to view progress information for enrolled courses (completed material counts and related status).

FR-36: Authenticated users shall be able to search courses and related records from the search page.

FR-37: Users shall be able to submit a course review with a rating.

5. Payments, Certificates, and Gamification

FR-38: The system shall support checkout for premium courses using Stripe when configured, or a simulation path when no secret key is set.

FR-39: The system shall be able to issue a completion certificate that a student can view or download.

FR-40: The system shall track gamification points or achievements based on completed academic activity.

6. Administration

FR-41: Administrators shall be able to view users and manage roles or account status according to implemented admin APIs.

FR-42: Administrators shall have access to a reports/overview workspace summarizing platform activity.

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

- NFR-1: Typical dashboard list pages (assignments, notifications, calendar) shall load under normal campus network conditions without blocking the rest of the shell.
- NFR-2: Average public page load on a development machine shall remain interactive after the Vite client is compiled.

3.2.2 Security Requirements

- NFR-3: Passwords shall be stored using bcrypt hashing and shall never be returned in API responses.
- NFR-4: Protected API routes shall require a valid JWT.
- NFR-5: Role-restricted actions (create assignment, grade submission, manage users) shall be enforced on the server, not only in the user interface.
- NFR-6: Production deployments shall serve the client and API over HTTPS.

3.2.3 Reliability & Availability

- NFR-7: The API shall handle invalid input with structured error responses rather than crashing the process.
- NFR-8: When local MongoDB is down, the server shall attempt an in-memory fallback so the demo can continue.
- NFR-9: For a persistent deployment, MongoDB backups are recommended; in-memory mode is for demonstration only.

3.2.4 Maintainability

- NFR-10: Server code shall be organized by routes, controllers, models, middleware, and services.
- NFR-11: Client feature screens shall be split by domain (academic, community, tracking, core) to keep the dashboard maintainable.

3.2.5 Scalability

- NFR-12: The architecture shall allow the API and MongoDB to be hosted separately so each can scale independently.
- NFR-13: Stateless JWT authentication shall allow more than one API instance behind a load balancer.

3.3 External Interface Requirements

3.3.1 User Interfaces

- UI-1: A responsive React interface for the public site (home, catalog, preview, about, contact, blog, login, register).

- UI-2: Role-specific dashboards under /app with a sidebar, header, and feature pages.
- UI-3: Forms for login, registration, course join, assignment create/submit, discussions, calendar events, and messaging.

3.3.2 Hardware Interfaces

- HI-1: None specific. The system runs on standard PCs, laptops, or cloud virtual machines. End users need a device with a modern browser.

3.3.3 Software Interfaces

- SI-1: MongoDB (or mongodb-memory-server) for persistence.
- SI-2: Stripe API for premium checkout when STRIPE_SECRET_KEY is present.
- SI-3: SMTP / Nodemailer for outbound mail when email environment variables are set.
- SI-4: Vite development server and a static host for the built React client.

3.3.4 Communication Interfaces

- CI-1: RESTful JSON APIs under /api for communication between the React client and Express server.
- CI-2: Authorization header carrying the JWT on authenticated requests.
- CI-3: CORS configured for the frontend origin (for example http://localhost:5173).

4. Technology Stack & Architectural Overview

4.1 MERN Stack Components

- MongoDB: document database for users, courses, enrollments, assignments, submissions, quizzes, announcements, discussions, messages, notifications, attendance, progress, reviews, calendar events, payments, certificates, and activity records.
- Express.js: Node.js framework that exposes REST routes, validation, file upload, and error handling.
- React.js: frontend library (React 19 with React Router) for the public site and dashboards, styled with a project stylesheet and Bootstrap utilities.
- Node.js: runtime that executes the API (server.js) and seed script.

Supporting libraries include Mongoose, JSON Web Token, bcryptjs, Axios, Vite, Recharts, Multer, PDFKit (certificates), and optional Stripe and Nodemailer.

5. Tentative Development Plan (Agile Methodology)

Development followed an Agile approach with short sprints. Each sprint delivered a usable increment of Melango, from the public site and authentication through academic features, communication, and polish.

Sprint 1 (Weeks 1-2): Core Setup & Authentication

- Initialize the MERN project structure (melango/client and melango/server).

- Configure Express, MongoDB connection, environment variables, and CORS.
- Implement registration, login, JWT auth, and password hashing.
- Build the public homepage, navigation, about, and contact shells.
- Create student, teacher, and admin role routing for /app.

Sprint 2 (Weeks 3-4): Courses, Enrollment & Dashboards

- Teacher course creation with enrollment codes.
- Student join-by-code and My Courses views.
- Public catalog and course preview.
- Role dashboards with KPIs and welcome header.
- Admin user and course overview.

Sprint 3 (Weeks 5-6): Academic Work & Assessment

- Learning materials with completion state.
- Assignment create, list, submit, and grade/feedback.
- Quizzes and results.
- Attendance recording.
- Progress analytics for students.

Sprint 4 (Week 7): Communication & Tracking

- Announcements and discussion forum.
- Private messaging.
- Notification create/list/read-all.
- Calendar with personal events and derived assignment/quiz deadlines.
- Search.

Sprint 5 (Weeks 8-9): Payments, Quality & Final Touches

- Reviews, certificates, gamification, and payment/checkout (including simulation mode).
- Public site visual polish (hero, journey, catalog, typography, solid purple buttons).
- Responsive dashboard shell and remaining UI fixes.
- Seed data and demo accounts for evaluation.
- SRS documentation.

6. Acceptance Criteria

The product is accepted for CSE470 demonstration when the following are true:

- A visitor can open the public site and navigate Home, All Courses, About, Contact, Blog, Login, and Sign Up.
- A new student or teacher can register and log in; an admin can log in with a seeded account.
- A teacher can create a course and share an enrollment code; a student can join that course.

- A teacher can create an assignment; a student can submit it; the teacher can grade it.
- A student can open Notifications and Calendar after academic activity and see relevant items when data exists.
- Discussion threads and replies work for a selected course for users who can open the Discussions page.
- Unauthorized users cannot call teacher/admin APIs successfully.
- The API starts with MongoDB or the in-memory fallback and reports the listening port.

7. Conclusion

This SRS provides a detailed outline for the design and development of “Melango” – a comprehensive learning management web application using the MERN stack. It supports a learner-centered approach through 20 core features: role management, course creation and enrollment, dashboards, assignments, submission, grading, feedback, announcements, discussions, quizzes, attendance, materials, calendar, notifications, messaging, progress analytics, search, admin control, and gamification. By following the outlined Agile sprints, the team delivered a usable, role-based LMS that can be run locally for demonstration.

8. Sprint Feature Descriptions

The following write-ups cover the 20 implemented Melango features, grouped by sprint in the WeHeal SRS style. Screenshots are not included. Evaluators can verify each feature in the running application.

SPRINT 1

Feature 1: Role Management

Supports Student, Teacher, and Admin roles with role-based permissions. Registration stores a chosen role (student or teacher). Admin accounts are seeded or promoted by an existing admin. After login, each role receives a different sidebar and is blocked from unauthorized API actions (for example, students cannot create courses or grade submissions).

Feature 2: Course Creation

Allows instructors to create and manage courses. A teacher opens Create course, enters the course details, and the server stores the course with an enrollment code. The teacher can later view and manage that course from My courses.

Feature 3: Course Enrollment

Students join courses using enrollment codes. The student enters a valid code from My courses. If the code matches and the student is not already enrolled, an enrollment record is created. Invalid or duplicate enrollments are rejected.

Feature 4: Dashboard

Provides an overview of courses, activities, and deadlines. After login, the role dashboard shows a welcome header, KPI-style counts (such as pending assignments), and shortcuts into courses, calendar, and other workspace tools.

SPRINT 2

Feature 5: Assignment Management

Teachers create, schedule, and manage assignments. For a selected course they add a title, description, due date, and total marks. Students see the assignment list for courses they joined, including due dates and status.

Feature 6: Assignment Submission

Students submit files, links, or text responses. In the dashboard they enter notes and an optional link. The API also accepts an optional file upload with the submission.

Feature 7: Online Grading System

Teachers grade submissions digitally. From Assignments they open Submissions, read the work, enter marks, and save. Students later see the graded status and awarded marks.

Feature 8: Feedback & Comments

Enables personalized feedback and communication on submitted work. Teachers add comments when grading. Students can see those comments with their marks.

SPRINT 3

Feature 9: Announcement Board

Share important notices and course updates. Teachers or admins post announcements. Students and other enrolled users read them from the Announcements page.

Feature 10: Discussion Forum

Supports academic discussions and Q&A. Users pick a course, start a thread with a topic and message, and reply to existing threads. The page is listed in the teacher sidebar; the same `/app/discussions` route is available to other roles.

Feature 11: Quiz & Exam Module

Create and manage quizzes and examinations. Teachers create quizzes on a course. Students attempt an available quiz and receive a recorded result.

Feature 12: Attendance Tracking

Record and monitor student attendance. Teachers record attendance against a course roster. Records are stored per student and date for later review.

SPRINT 4

Feature 13: Learning Materials Repository

Store notes, PDFs, videos, and learning resources. Teachers add materials to a selected course. Students open Materials, view the list, and can mark an item

completed.

Feature 14: Calendar & Deadline Tracker

Track academic events and deadlines. Calendar lists stored events plus derived assignment due dates and quiz close dates. Students can add a personal event. Course-linked events may be added only by the course owner.

Feature 15: Notification System

Send reminders and important alerts. Creating an assignment notifies enrolled students (for example, “New assignment posted”). Users open Notifications to read items, mark one read, or mark all read. A header bell opens this page.

Feature 16: Messaging System

Direct communication between students and instructors. The Messages page lists contacts. Selecting a contact loads the conversation. Sending a message stores it and refreshes the thread.

SPRINT 5

Feature 17: Progress Analytics

Monitor academic performance and engagement. Students open Progress to see completion information for enrolled courses, including completed material counts from their activity.

Feature 18: Search & Filter System

Quickly locate courses, materials, and assignments. Authenticated users search from the Search page and can filter results instead of walking every sidebar item.

Feature 19: Admin Control Panel

Manage users, permissions, courses, and platform reports. An admin can open Users to change roles, inspect Courses, post announcements, and use Reports for a high-level view of platform activity.

Feature 20: Gamification System

Uses badges, achievements, points, and leaderboards to increase engagement. Completing assignments, quizzes, and related work awards points and badges. Students open Achievements to see level, badge list, and a leaderboard.

Appendix A - How to Run the Application

API (terminal 1):

```
cd melango/server then npm install then npm start
```

Client (terminal 2):

```
cd melango/client then npm install then npm run dev
```

Open the Vite URL (<http://localhost:5173> or the next free port). The client environment points at <http://localhost:5001/api>.

Appendix B - Demo Accounts

Role	Email	Password
Student	student@melango.com	Student123!
Teacher	teacher@melango.com	Teacher123!
Admin	admin@melango.com	Admin123!

If the API uses in-memory MongoDB, restarting the server clears data. Log in again with the seeded demo users. Optional: `npm run seed` in the server folder loads a richer dataset when a persistent MongoDB is available.

Appendix C - Team

Student ID	Name	Contribution focus
23301489	Tahsin Pathan	LMS product, client UX, integration
23301334	Shakeel	Features and documentation
22299386	Tabassum Islam	Features and documentation
23101108	Ashrafi Tahmid	Features and documentation